MACHINE LEARNING

A Hitchhiker’s Guide to Sentiment Analysis using Naive-Bayes Classifier

Anwesan De

May 14, 2021 12 min read

A tutorial to one of the most elegant and beauti
Natural Language Processing.

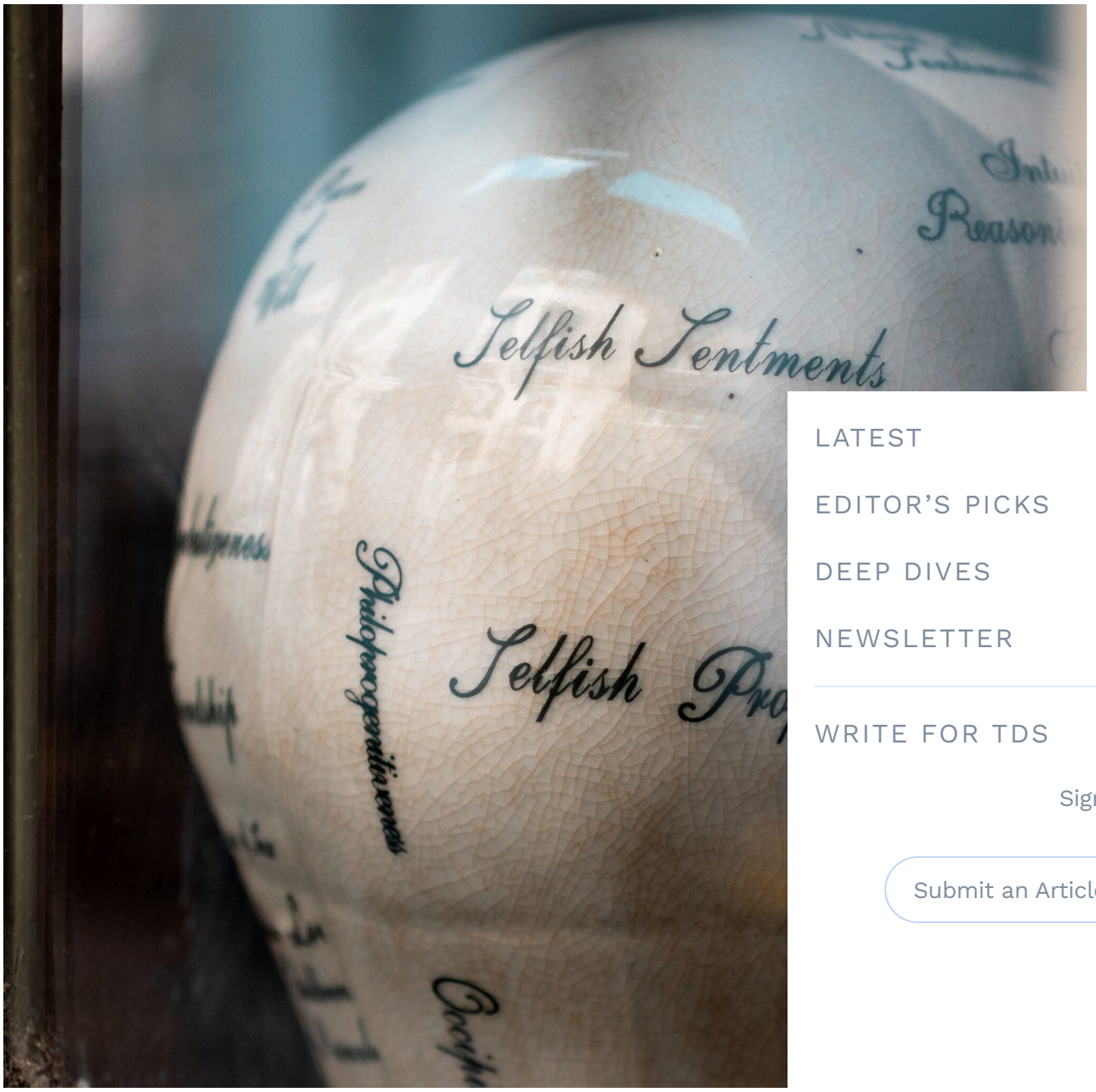

image from Unsplash

Classification lies at the heart of Machine Learr
Intelligence. Recognizing voices or faces or what
on a daily basis all come under the umbrella of
Now coming to Naive Bayes , for anyone starting
of Natural Language Processing , this is the first
towards this goal ;that is why it is of paramount
understand how to implement this and what's g

bonnet . In this article we are going to learn how to implement Naive-Bayes from scratch and use it for Sentiment analysis.

INDEX:

1. **SENTIMENT ANALYSIS**
2. **TERMINOLOGY**
3. **NAIVE-BAYES THEOREM**
4. **DERIVATION**
5. **TRAINING THE MODEL**
6. **SOLVED EXAMPLE**
7. **CONCLUSION**

LATEST
EDITOR'S PICKS
DEEP DIVES
NEWSLETTER
WRITE FOR TDS
Sign in
Submit an Article

SENTIMENT ANALYSIS

Now sentiment analysis as the name suggests is
task where we classify the sentiment of the sta
simply the emotion one particular statement is
whether its positive or negative , sad or sarcasti
wholesome and kind . Let's elaborate this point
examples;

Let's say there's a new restaurant in Town and y
friend decided to have dinner there and experie
You really enjoyed yourself and when your frienc
opinion, you say something along the lines :

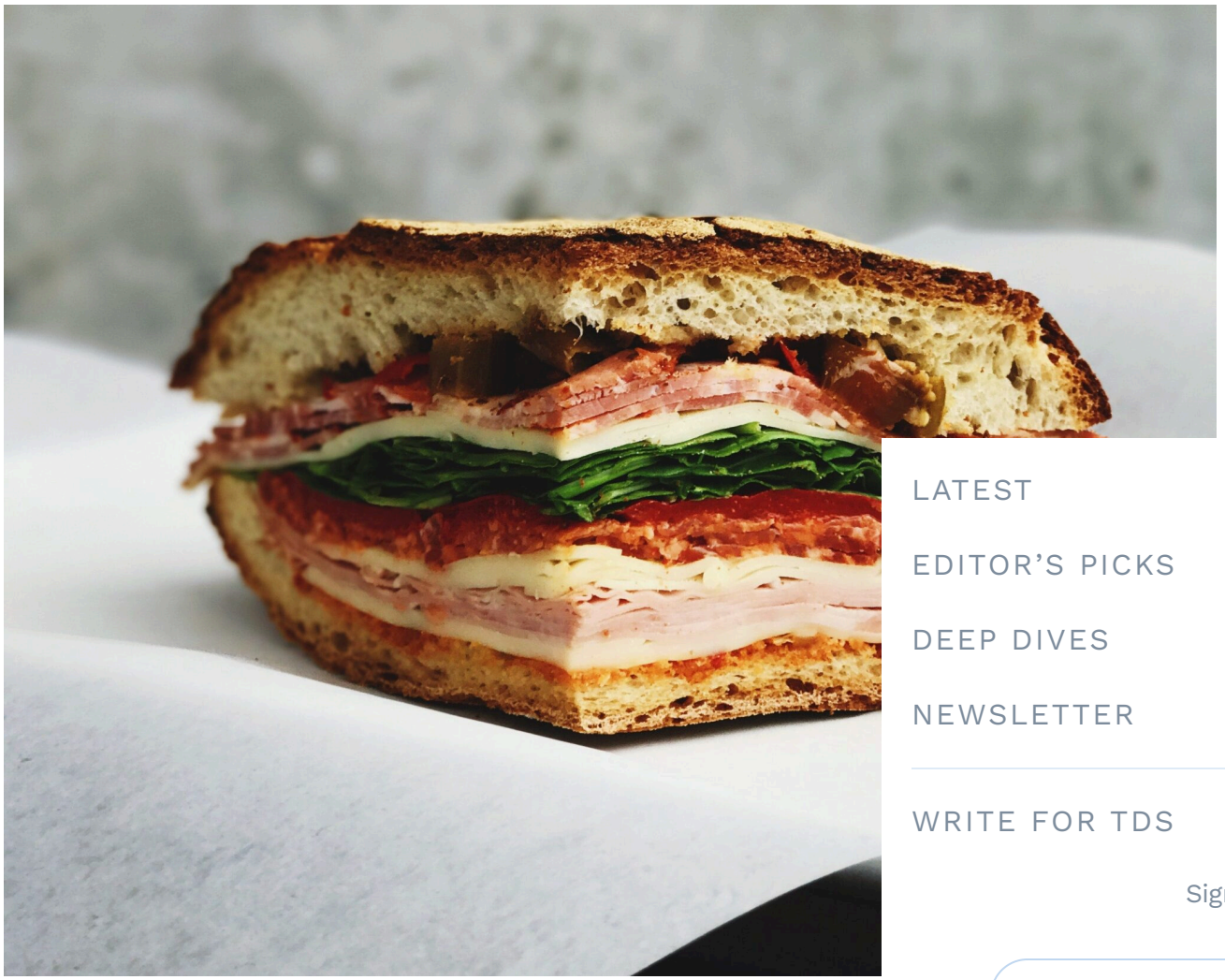

image from Unsplash

1." This place is great . The food is delicious and quite enjoyable"

Now your friend has contrary beliefs . This new
his liking . So when you in turn ask for his opini

2."*This place is pathetic. The food is horrible ana quite overpowering for me and makes me really*

I think we can all agree that statement 1 expres sentiment while statement 2 represents a negat an emotion. What's also important for us to not

EXACTLY gave us the clues which allowed us to classify the sentences into positive or negative .

The Answer? The cues...

if you look closely... There are certain words in the sentences that dictate the emotion . Positive words like ***great, delicious, enjoyable*** etc. make the sentence positive while words like ***pathetic , horrible , uncomfortable*** etc. Make th negative . In fact if we just replace these specia sentence it completely changes its meaning. Le

Sentiment	Sente
positive	*"Godfather is t all time"*
negative	*" Godfather is of all time"*
positive	*" I love Led Ze*
negative	*" I despise Led*

image by author

In each of these examples , if you notice, by cha word changes the entire flavor of the sentence the sentence remains exactly the same . This is Sentiment Analysis . Also we are going to limit c **binary classification** , classifying the sentences or negative.

TERMINOLOGY

Now before digging into mathematics let's first talk about terminology . What we are going to do here falls under **Supervised Machine Learning** ; which means that we will be provided with training inputs and each of these inputs will be associated with its correct output . Now it's the job of your model to make sense of this data , observe and analyze the relationship between the given inputs and outputs and finally predict with reasonable accuracy the output ; gi

Going further ; we usually denote input as x and
where y ∈[y, yb, yc,.....yn] classes and it's your m
predict which output class a particular input x b
in this case we will be dealing with words and s
is a slight change in terminology . Our input will
document (sentences basically) consisting of a r
features and our output would be '**c**' for class. H
represent either positive(for positive sentiment)
negative sentiment).

So finally we will get an input d and our model h
predict which class , 'c' , it belongs to .

Now that we are done with the nitty gritty and t
let's start with the theorem .

NAIVE-BAYES THEOREM

Let's first look into **Bayes Theorem:**

P(A | B) = P(B | A) * P(A) / P(B) ->(1)

Let's Look into the terms:

- P(A | B) = Probability of event A happening given that event B happens
- P(B | A) = Probability of event B happening given that event A happens
- P(A) = Probability of event A happening
- P(B) = Probability of event B happening

The Bayes Theorem thus gives us a way to find t
Probability . Bayes Theorem lies in the heart of t
theorem.

Now we are in a position to describe Multinomia
Theorem. As the name suggests, this theorem u
Classifier with a simplified assumption about ho
interact .

One of the most important assumptions we hav
Naive-Bayes is called the **bag-of-words .** It impl
algorithm really cares about is the word and it's
how many times the word has appeared in our c
of the word in our sentence(document) does no
only have to keep track of the number of times
appeared in our document . That's it .

Let me explain with an example:

" *Tea makes me happy. Black Tea, Green Tea, Mil*
matter what kind it is ; as long as it's tea I am s
grown up drinking tea and every sip reminds me
days"

LATEST
EDITOR'S PICKS
DEEP DIVES
NEWSLETTER
WRITE FOR TDS
Sign in
Submit an Article

Let’s analyze it further:

WORD	COUNT
Tea	6
happy	1
satisfied	1
I	2
as	[illegible]

image by author

This is basically what bag-of-words concept boi
does not matter where the words *tea* , *I*, *happy* ,
have been used in the sentences , all that matte
frequency.

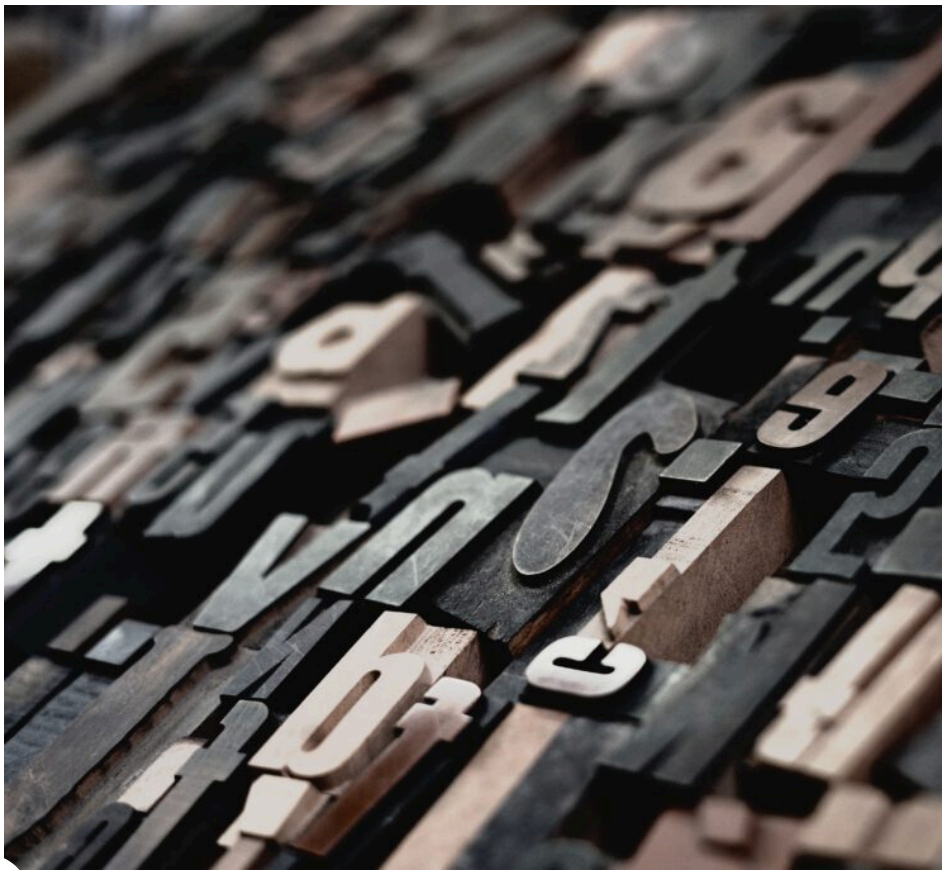

image from Unsplash

DERIVATION

Now let's try to formulate this mathematically.

If you may recall, our main goal was to find the class (whether positive or negative sentiment) given a particular sentence(document). So we can approach this problem this way:

1. Suppose we have a set of possible classes (
2. We find the probability of a document being
   class.

So essentially conditional probability of a class g

.

3. We iterate over all the classes and find whic
   maximum conditional probability ; giving us

Combining all the steps together we get :

$$\hat{c} = \underset{c \in C}{\operatorname{argmax}}\ P(c|d)$$

image by author

Here the term ĉ denotes the class with the max

.

We were already introduced to the Bayes Theore
can plug in the formula for conditional probabili

LATEST
EDITOR'S PICKS
DEEP DIVES
NEWSLETTER
WRITE FOR TDS
Sign in
Submit an Article

$$\hat{c} = \underset{c \in C}{\operatorname{argmax}}\ P(c \mid d) = \underset{c \in C}{\operatorname{argmax}}\ P(d \mid c)\, P(c) / P(d) \qquad (3)$$

image by author

We can simplify this equation even more . While iterating over the classes our document of course does not change , only the class changes ; so we can safely remove the P(d) from our denominator without causing any major problems. So our mo thus becomes:

$$\hat{c} = \underset{c \in C}{\operatorname{argmax}}\ P(c|d) = \underset{c \in C}{\operatorname{argmax}}\ P(d|c)P(c)$$

image by author

The term P(d|c) is called **likelihood probability**

The second term P(c) is called **prior probability**

We can simplify it even further by dividing each collection of features *f1 , f2, f3, ...fn.*

$$\hat{c} = \underset{c \in C}{\operatorname{argmax}}\ P(f_1, f_2, \ldots, f_n \mid c)\, P(c)$$

image by author

At this point of our derivation we will make a ve assumption . We will assume that the probabilit f given is class is **independent** of each other. Th step and it reduces the time complexity of our p margin. Let's understand that a bit more.

LATEST
EDITOR'S PICKS
DEEP DIVES
NEWSLETTER
WRITE FOR TDS
Sign in
Submit an Article

If two events X and Y are independent of each other then the probability of the events occurring together (P(X and Y)) becomes:

$P(X \cap Y) = P(X) * P(Y)$

which means:

$P(f 1 | c \cap f 2 | c) = P(f 1 | c) * P(f 2 | c)$

We can simplify eqt(5) even further! Also , assur
are independent from each other we do not hav
account how each feature is related to one anot
probability of one feature occurring given anoth
saves us a lot of computing power .

Thus our final equation becomes:

$P(f 1 , f 2 , ..., f n |c) = P(f 1 |c) \cdot P(f 2 |c) \cdot ... \cdot P($

or

$$\hat{c} = \underset{c \in C}{\operatorname{argmax}} P(c) \prod_{f \in F} P(f |c)$$

image by author

Now of course the features in a sentence would
so if we replace the features in our equation wit
at the i-th position we can re-frame our equatic

$$\hat{c} = \underset{c \in C}{\operatorname{argmax}} P(c) \prod_{i \in positions} P(w_i |c)$$

image by author

Phew!!!! we are finally done with the derivation. Now, lets move onto how we can apply this concept in a practical problem .

TRAINING THE MODEL

1. **Calculate the Prior Probability** . We will first find the number of documents belonging to each class . Finding the percentage of the documents in each class will give us the required prior probability.

Let's assume the number of documents in class

Total number of documents is assumed to be *N*

So , P(c) = *Nc / Ntotal* ->(9)

2. **Calculate the Likelihood Probability**. This is slightly tricky . Our main goal is to find the f the word wi appears among all words in all class c. We first concatenate all documents into one big "category c" text. Then we use t wi in this concatenated document to give th probability .

$$P(w_i \mid c) = count(w_i , c) / \sum_{w \in V} count(w_i, c)$$

image by author

Here V is for Vocabulary which is a collection of documents irrespective of class

LATEST
EDITOR'S PICKS
DEEP DIVES
NEWSLETTER
WRITE FOR TDS
Sign in
Submit an Article

We will however face a very unique problem at this point . Suppose the document we have as input is ,

d = "*I loved that movie*"

The word "loved" is only present in the positive class and no examples of "loved" is present in the negative class input .Now from eqt(8) we have to find the probability by multiplying the likelihood probability for each class.If we calcula probability for the word "loved" for the class "ne

P("loved" | "negative") = 0

Now if we plug in this value in eqt(8) the entire class "negative" becomes zero ; no matter what are .

To combat this problem we will introduce an ad **Smoothing Coefficient** , to both the numerator a denominator . Our equation will be modified as

$$P(w_i \mid c) = \text{count}(w_i, c) + a \,/\, \sum_{w \in V} (\text{count}(w_i, c) +$$

or :

$$P(w_i \mid c) = \text{count}(w_i, c) + a \,/\, \sum_{w \in V} (\text{count}(w_i, c))$$

image by author

Here a is the Laplace smoothing coefficient . We s value to be 1.

LATEST
EDITOR'S PICKS
DEEP DIVES
NEWSLETTER
WRITE FOR TDS
Sign in
Submit an Article

3. **Plug in the prior and the likelihood probability in eqt(8).**

Now that we have calculated our prior and likelihood probability we can simply go ahead and plug it in .

There are few ways we can optimize this process though:

a. **Using Logarithm**: if we apply log on both sides of eqt(8) we can convert the equation to a linear function of the would increase efficiency quite a lot.

This is the original equation we had:

$$\hat{c} = \underset{c \in C}{\operatorname{argmax}}\, P(c) \prod_{i \in positions} P(w_i|c)$$

image by author

Now if we apply Logarithm on both sides we get

$$\hat{c} = \underset{c \in C}{\operatorname{argmax}}\, \log P(c) + \sum_{i \in positions} (\log P(w_i|c)$$

image by author

b. **Stop Words**: Words like *the, a , an , was , whe*

not usually contribute to the sentiment of the s can remove them entirely to streamline our mo

c. **Unknown Words:** Every time you face a word in the test dataset but absent in the vocabulary training data , it is advisable to drop the words consider them in the probability calculations.

LATEST
EDITOR'S PICKS
DEEP DIVES
NEWSLETTER
WRITE FOR TDS
Sign in
Submit an Article

d. **Binary Multinomial Naive-Bayes** : This is a slightly modified version of the multinomial Naive-Bayes. Here we are going to place more importance on whether a word is present or not than its frequency . As we have already seen a single word can bring about a massive change in the sentiment of the sentence and thus it would be a logical way to disregard how many times that particular word appeared in a sentence and concentrate whether that particular word is present or not in the doc

SOLVED EXAMPLE

Finally , now that we are familiar with Naive Bay can implement this knowledge in an example .

Training Dataset:

DOCUMENT	CLASS
" Boring and Predictable"	-
" Excellent movie"	+
"Extremely mediocre"	-
" A pathetic attempt at a romcom"	-
" Good movie with great actors"	+
"Fantastic job!"	+

image by autor

Test Dataset:

DOCUMENT
“ Great movie !!”
“ This is boring and pathetic.”

image by author

This is a fictitious dataset on movie reviews. The movie reviews have been divided into positive and negative cla

Let’s Solve the problem: (we will consider smoo *α,* as 1)

For the first test case:

Prior probability:

P(c = ‘positive’) = 3/6 = 1/2 , P(c = ‘negative’) = 3,

Likelihood probability:

P(‘Great’ | c = ‘Positive’) = 1 + 1 / (9 + 19) = 0.071

P(‘movie’ | c = ‘Positive’) = 2 + 1 /(9 + 19) = 0.1071

P(‘Great’ | c = ‘Negative’) = 0 + 1/(10 + 19) = 0.03

P(‘movie’ | c = ‘Negative’) = 0 + 1/(10 + 19) = 0.03

Finally we apply our findings in eqt(8) and retur probability :

P(c = ‘positive’) *P(‘Great’ | c = ‘Positive’)* P(‘movie’ | c = ‘Positive’) = 0.5 *0.0714* 0.1071 = 0.00382

P(c = ‘negative’) *P(‘Great’ | c = ‘Negative’)* P(‘movie’ | c = ‘Negative’) = 0.5 *0.0344* 0.0344 = 0.000591

Thus we can say the test case belongs to ***positive*** class.

For the second test case:

Prior probability:

P(c = ‘positive’) = 3/6 = 1/2 , P(c = ‘negative’) = 3/

In this example we encounter unknown words : not consider them in our calculations.

Likelihood probability:

P(‘boring’ | c = ‘Positive’) = 0 + 1 / (9 + 19) = 0.03

P(‘pathetic’ | c = ‘Positive’) = 0 + 1 /(9 + 19) = 0.0

P(‘and’ | c = ‘Positive’) = 0 + 1 /(9 + 19) = 0.03571

P(‘boring’ | c = ‘Negative’) = 1 + 1 / (10+ 19) = 0.06

P(‘pathetic’ | c = ‘Negative’) = 1 + 1 /(10 + 19) = 0.

P(‘and’ | c = ‘Negative’) = 1 + 1 /(10 + 19) = 0.0689

Finally we apply our findings in eqt(8) and retur
probability :

LATEST
EDITOR’S PICKS
DEEP DIVES
NEWSLETTER
WRITE FOR TDS
Sign in
Submit an Article

P(c = ‘positive’) *P(‘boring’ | c = ‘Positive’)* P(‘pathetic’ | c = ‘Positive’) *P(‘and’ | c = ‘Positive’) = 0.5* 0.03571 *0.03571* 0.03571 = 0.00002277

P(c = ‘negative’) *P(‘Great’ | c = ‘Negative’)* P(‘movie’ | c = ‘Negative’) = 0.5 *0.0689* 0.0689 * 0.0689 = 0.00016354

Thus we can say the test case belongs to ***negative*** class.

CONCLUSION

Sentiment analysis is widely used in social medi
market research etc. It is one of the most impo
Natural Language Processing . Naive – Bayes Cla
great first step towards Natural Language proce
you enjoyed reading this article .

The python code implementing Binary Multinom
Classifier can be found in my github repo . The
been included in the repo.

WRITTEN BY

Anwesan De

See all from Anwesan De

Basics Machine Learning Machine Learning